

ECON 609: Projection

Minsu Chang

Georgetown University

Lecture # 6

Introduction

- ▶ We come back to our function equation:

$$\mathcal{H}(d) = \mathbf{0}$$

- ▶ Projection methods solve the problem by specifying:

$$d^n(x, \theta) = \sum_{i=0}^n \theta_i \Psi_i(x)$$

We pick a basis $\{\Psi_i(x)\}_{i=0}^{\infty}$ and "project" $\mathcal{H}(\cdot)$ against the basis to find the θ_i 's.

- ▶ How?

Some Comments

- ▶ We may want to approximate different objects d : for instance, a decision rule, a value function, or an expectation.
- ▶ We will work with linear combinations of basis functions.
Why? The theory of nonlinear approximations is not yet as developed as the linear case.

Basic Algorithm

- (1) Define n known linearly independent functions $\psi_i : \Omega \rightarrow \mathbb{R}^m$ where $n < \infty$. We call $\psi_1(\cdot), \psi_2(\cdot), \dots, \psi_n(\cdot)$ **basis functions**.
- (2) Define a vector of parameters $\theta = [\theta_1, \theta_2, \dots, \theta_n]$.
- (3) Define a combination of the basis functions and the θ 's:

$$d^n(\cdot|\theta) = \sum_{i=1}^n \theta_i \psi_i(\cdot)$$

- (4) Plug $d^n(\cdot|\theta)$ into $\mathcal{H}(\cdot)$ to find the **residual equation**:

$$R(\cdot|\theta) = \mathcal{H}(d^n(\cdot|\theta))$$

- (5) Find the value of $\hat{\theta}$ that make the residual equation as close to $\mathbf{0}$ as possible given some objective function $\rho : J^1 \times J^1 \rightarrow J^2$:

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^n} \rho(R(\cdot|\theta), \mathbf{0})$$

Two Issues

- ▶ We need to decide:
 - (1) Which basis we use?
 - (2) How do we "project?"
- ▶ Different choices in (1) and (2) will result in slightly different projection methods.
- ▶ We will look at Chebyshev polynomials, which provide orthogonal basis functions.

Definition of Chebyshev Polynomials I

- ▶ Recursive definition:

$$T_0(x) = 1$$

$$T_1(x) = x$$

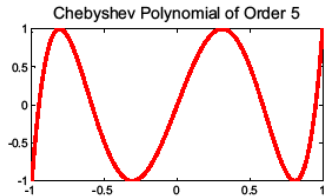
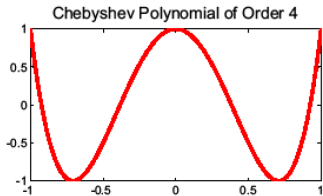
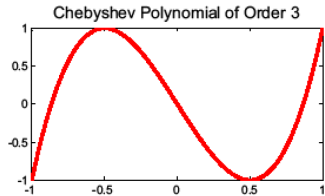
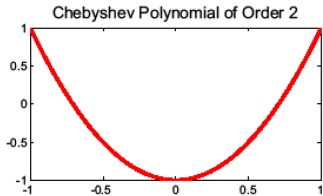
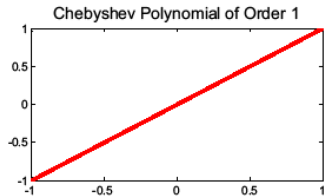
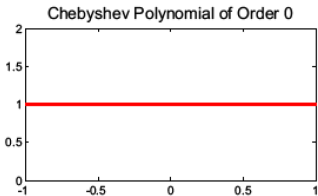
$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \quad \text{for a general } n$$

- ▶ The first few polynomials are then

$$1, x, 2x^2 - 1, 4x^3 - 3x, 8x^4 - 8x^2 + 1, \dots$$

- ▶ The n zeros of the polynomial $T_n(x_k) = 0$ are given by:

$$x_k = \cos \frac{2k-1}{2n} \pi, \quad k = 1, \dots, n$$



Definition of Chebyshev Polynomials II

- Explicit definition:

$$\begin{aligned}T_n(x) &= \cos(n \arccos x) \\&= \frac{1}{2} \left(z^n + \frac{1}{z^n} \right) \quad \text{where } \frac{1}{2} \left(z + \frac{1}{z} \right) = x \\&= \frac{1}{2} \left((x + (x^2 - 1)^{0.5})^n + (x - (x^2 - 1)^{0.5})^n \right)\end{aligned}$$

Some Comments

- ▶ The domain of the Chebyshev polynomials is $[-1, 1]$. Since our state space is, in general, different, we use a linear translation from $[a, b]$ into $[-1, 1]$:

$$2\frac{x - a}{b - a} - 1$$

- ▶ Chebyshev polynomials have a nice property as the approximation error becomes arbitrarily small with the order of Chebyshev polynomial $\rightarrow \infty$.

Example

```
function R = chebroots(n,kmin,kmax)
% Compute Roots of a Chebyshev Polynomial of degree n

C = [];

for k = 1:n
    auxr = cos(((2*k-1)/(2*n))*pi);
    C = [C;auxr];
end

% Map to Capital Grid

R = kmin+0.5*(kmax-kmin)*(1+C);

end
```

Multidimensional Problems

- ▶ Chebyshev polynomials are defined on $[-1, 1]$.
- ▶ However, most problems in economics are multidimensional.
- ▶ How do we generalize the basis?
- ▶ Curse of dimensionality.

Tensors

- ▶ Assume we want to approximate $F : [-1, 1]^d \rightarrow \mathbb{R}$.
- ▶ Let T_j denote the Chebyshev polynomial of degree $j = 0, 1, \dots, \kappa$.
- ▶ We can approximate F with tensor product of Chebyshev polynomials of degree κ :

$$\hat{F}(x) = \sum_{n_1=0}^{\kappa} \dots \sum_{n_d=0}^{\kappa} \zeta_{n_1, \dots, n_d} T_{n_1}(x_1) \dots T_{n_d}(x_d)$$

- ▶ Advantage: If the one-dimensional basis is orthogonal in a norm, the tensor basis is orthogonal in the product norm.
- ▶ Disadvantage: The number of elements increases exponentially...

Smolyak's Algorithm I

- ▶ Define $m_1 = 1$ and $m_i = 2^{i-1} + 1$, $i = 2, \dots$
- ▶ Define $\mathcal{G}^i = \{x_1^i, \dots, x_{m_i}^i\} \subset [-1, 1]$ as

$$x_j^i = -\cos\left(\frac{\pi(j-1)}{m_i-1}\right), \quad j = 1, \dots, m_i$$

with $\mathcal{G}^1 = \{0\}$. It is crucial that $\mathcal{G}^i \subset \mathcal{G}^{i+1}, \forall i = 1, 2, \dots$

- ▶ Example:

$$i = 1, m_i = 1, \mathcal{G}^i = \{0\}$$

$$i = 2, m_i = 3, \mathcal{G}^i = \{-1, 0, 1\}$$

$$i = 3, m_i = 5, \mathcal{G}^i = \left\{-1, -\cos\left(\frac{\pi}{4}\right), 0, -\cos\left(\frac{3\pi}{4}\right), 1\right\}$$

Smolyak's Algorithm II

- For $q > d$, define a sparse grid

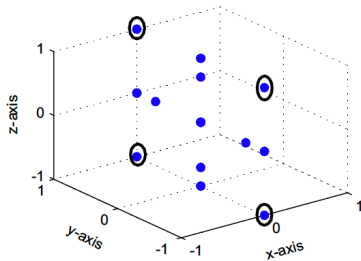
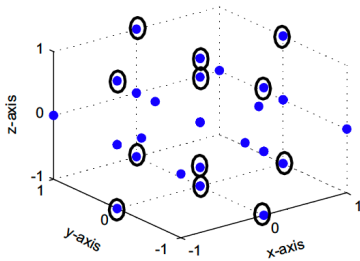
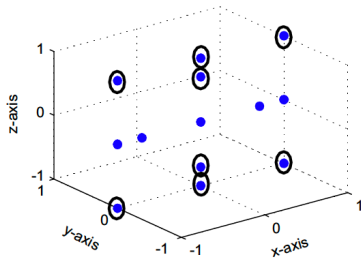
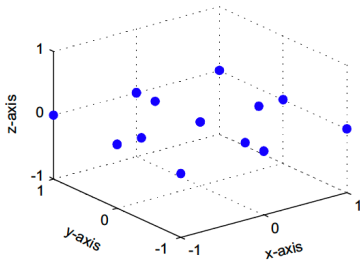
$$\mathcal{H}(q, d) = \bigcup_{q-d+1 \leq |i| \leq q} (\mathcal{G}^{i_1} \times \dots \times \mathcal{G}^{i_d}),$$

where $|i| = i_1 + \dots + i_d$. The number q defines the size of the grid and thus the precision of the approximation.

- For example, let $q = d + 2 = 5$:

$$\mathcal{H}(5, 3) = \bigcup_{3 \leq |i| \leq 5} (\mathcal{G}^{i_1} \times \dots \times \mathcal{G}^{i_d}).$$

$$\begin{array}{lll} \mathcal{G}^3 \times \mathcal{G}^1 \times \mathcal{G}^1, & \mathcal{G}^1 \times \mathcal{G}^3 \times \mathcal{G}^1, & \mathcal{G}^1 \times \mathcal{G}^1 \times \mathcal{G}^3 \\ \mathcal{G}^2 \times \mathcal{G}^2 \times \mathcal{G}^1, & \mathcal{G}^2 \times \mathcal{G}^1 \times \mathcal{G}^2, & \mathcal{G}^1 \times \mathcal{G}^2 \times \mathcal{G}^2 \\ \mathcal{G}^2 \times \mathcal{G}^1 \times \mathcal{G}^1, & \mathcal{G}^1 \times \mathcal{G}^2 \times \mathcal{G}^1, & \mathcal{G}^1 \times \mathcal{G}^1 \times \mathcal{G}^2 \\ \mathcal{G}^1 \times \mathcal{G}^1 \times \mathcal{G}^1 \end{array}$$



Smolyak's Algorithm III

- ▶ Number of points for $q = d + 2$:

$$1 + 4d + 4 \frac{d(d-1)}{2}$$

- ▶ Largest number of points along one dimension:

$$i = q - d + 1$$

$$m_i = 2^{q-d} + 1$$

- ▶ Rectangular grid (exponentially increasing!):

$$\left(2^{q-d} + 1\right)^d$$

Smolyak's Algorithm IV

- ▶ With the Smolyak's algorithm, the number of points increases polynomially in the number of dimensions.

Size of the Grid for $q = d + 2$			
d	$2^{q-d} + 1$	$\#\mathcal{H}(q, d)$	$[2^{q-d} + 1]^d$
2	5	13	25
3	5	25	125
4	5	41	625
5	5	61	3, 125
12	5	313	244, 140, 625

- ▶ Reference: [Malin et al. \(2010\)](#)

Choosing the Objective Function

- ▶ We need to decide:

(1) Which basis we use? (Chebyshev polynomials)

(2) How do we "project?"

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^n} \rho(\mathcal{H}(d^n(\cdot|\theta)), \mathbf{0})$$

- ▶ The most common answer to (2) is given by a **weighted residual**.
- ▶ This set of techniques propose to get the residual close to **0** in the weighted integral sense.

Choosing the Objective Function

- ▶ Given some weight functions $\phi_i : \Omega \rightarrow \mathbb{R}^m$:

$$\rho(R(\cdot|\theta), \mathbf{0}) = \begin{cases} 0 & \text{if } \int_{\Omega} \phi_i(x) R(\cdot|\theta) dx = \mathbf{0}, \quad i = 1, \dots, n \\ 1 & \text{otherwise} \end{cases}$$

- ▶ Then the problem is to choose the θ that solve the system of equations:

$$\int_{\Omega} \phi_i(x) R(\cdot|\theta) dx = \mathbf{0}, \quad i = 1, \dots, n$$

Some Comments

- ▶ With the approximation of d by some basis functions ψ_i and the definition of some weight functions $\phi_i(\cdot)$, we have transform a rather intractable functional equation problem into the nonlinear equations.
- ▶ The solution of this system can be found using standard methods, as a Newton for relatively small problems or a conjugate gradient for bigger ones.
- ▶ Issue: We have different choices for a weight function.

Weight Function I: Least Squares

► $\phi_i(x) = \frac{\partial R(x|\theta)}{\partial \theta_i}.$

- This choice is motivated by the solution of the variational problem:

$$\min_{\theta} \int_{\Omega} R^2(\cdot|\theta) dx$$

with first order condition:

$$\int_{\Omega} \frac{\partial R(x|\theta)}{\partial \theta_i} R(\cdot|\theta) dx = \mathbf{0}, \quad i = 1, \dots, n$$

- Variational problem is mathematically equivalent to a standard regression problem in econometrics.

Weight Function II: Collocation

- ▶ $\phi_i(x) = \delta(x - x_i)$ where δ is the dirac delta function and x_i are the collocation points.
- ▶ This method implies that the residual function is zero at the n collocation points.
- ▶ A systematic way to pick collocation points is to use a density

function:

$$\mu_\gamma(x) = \frac{\Gamma(\frac{3}{2} - \gamma)}{(1 - x^2)^\gamma \pi^{\frac{1}{2}} \Gamma(1 - \gamma)}, \quad \gamma < 1$$

and find the collocation points as the x_j , $j = 0, \dots, n - 1$ solutions to:

$$\int_{-1}^{x_j} \mu_\gamma(x) dx = \frac{j}{n}$$

- ▶ For $\gamma = 0$, the density function implies equispaced points.

Weight Function III: Orthogonal Collocation

- ▶ Variation of the collocation method:
 - (1) Basis functions are a set of orthogonal polynomials.
 - (2) Collocation points are given by the roots of the n^{th} polynomial.
- ▶ When we use Chebyshev polynomials, their roots are the collocation points implied by $\mu_{\frac{1}{2}}(x)$.
- ▶ Surprisingly good performance!

Guideline for PS2

- ▶ Obtain the collocation points for capital, k^{col}
- ▶ Find the coefficients which minimizes the residuals evaluated at the collocation points (e.g. fsolve)
 - ▶ Residuals are from the Euler equation

$$\frac{1}{c} = 0.96 \mathbb{E} \left[\frac{\alpha' e^{z'} (k')^{\alpha' - 1} (l')^{1 - \alpha'} + 0.9}{c'} \right]$$

- ▶ Approximate labor supply with Chebyshev polynomials

$$l(k^{col}, z, \alpha) = \sum_{i=1}^n \theta_i(z, \alpha) \cos(n \arccos(k^{col}))$$

- ▶ Given the policy function for labor and state variables, we can back out consumption.

Example

```
function R = residueq(theta,kgrid,kmin,kmax,zgrid,agrid,beta,PZ,PA)

R = zeros(length(zgrid),length(kgrid),length(agrid));

for m = 1:length(agrid)
    for t = 1:length(zgrid)
        for p = 1:length(kgrid)
            l = 0;
            kt = 2*(kgrid(p,1)-kmin)/(kmax-kmin)-1;
            for i = 1:length(kgrid)
                l = l+theta(t,i,m)*cos((i-1)*acos(kt));
            end
            c = (1-agrid(1,m))*exp(zgrid(1,t))*kgrid(p,1)^(agrid(1,m))/(l^(1+agrid(1,m)));
            kp = exp(zgrid(1,t))*kgrid(p,1)^(agrid(1,m))*l^(1-agrid(1,m))+0.9*kgrid(p,1)-c;
        end
    end
end
```

- ▶ ... Try it yourself!
- ▶ Collocation points k^{col} will be the input of the function above.